

Федеральное государственное автономное образовательное учреждение высшего образования «Национальный исследовательский университет ИТМО»

Факультет Программной инженерии и Компьютерной техники

Лабораторная работа №2
Численное решение нелинейных уравнений и систем
Вариант 5

Выполнила:
Косогова Мария Александровна
Группа: Р3206
Проверил:
Зинченко Антон Андреевич,
Преподаватель практики.

Санкт-Петербург 2026

Содержание

ЗАДАНИЕ	3
ОСНОВНЫЕ ЭТАПЫ ВЫПОЛНЕНИЯ	5
1. ВЫЧИСЛИТЕЛЬНАЯ РЕАЛИЗАЦИЯ ЗАДАЧИ	7
Часть 1. Решение нелинейного уравнения	7
часть 2. Решение системы нелинейных уравнений	10
2. ПРОГРАММНАЯ РЕАЛИЗАЦИЯ	12
Часть 1. Нелинейные уравнения	12
часть 2. Системы нелинейных уравнений	15
ВЫВОД	20

Задание

Лабораторная работа состоит из двух частей: вычислительной и программной.

1. Вычислительная реализация задачи

Часть 1. Решение нелинейного уравнения

$$-2,7x^3 - 1,48x^2 + 19,23x + 6,35$$

Крайний правый корень – Метод хорд

Крайний левый корень – Метод простой итерации

Центральный корень – Метод секущих

- 1) Отделить корни заданного нелинейного уравнения графически (вид уравнения представлен в табл. 6)
- 2) Определить интервалы изоляции корней.
- 3) Уточнить корни нелинейного уравнения (см. табл. 6) с точностью $\varepsilon=10^{-2}$
- 4) Используемые методы для уточнения каждого из 3-х корней многочлена представлены в таблице 7.
- 5) Вычисления оформить в виде таблиц (1-5), в зависимости от заданного метода. Для всех значений в таблице удерживать **3 знака** после запятой.
 - 5.1) Для метода половинного деления заполнить таблицу 1.
 - 5.2) Для метода хорд заполнить таблицу 2.
 - 5.3) Для метода Ньютона заполнить таблицу 3.
 - 5.4) Для метода секущих заполнить таблицу 4.
 - 5.5) Для метода простой итерации заполнить таблицу 5. **Проверить условие сходимости метода на выбранном интервале.**

Часть 2. Решение системы нелинейных уравнений

$$\begin{cases} tg(xy + 0,3) = x^2 \\ 0,9x^2 + 2y^2 = 1 \end{cases}$$

Метод Ньютона

- 1) Отделить корни заданной системы нелинейных уравнений графически (вид системы представлен в табл. 8).
- 2) Используя указанный метод, решить систему нелинейных уравнений с точностью до 0,01.
- 3) Для метода простой итерации проверить условие сходимости метода.

2. Программная реализация

Для нелинейных уравнений:

Метод половинного деления

Метод Ньютона

Метод простой итерации

- 1) Все численные методы (см. табл. 9) должны быть реализованы в виде отдельных подпрограмм/методов/классов.
- 2) Пользователь выбирает уравнение, корень/корни которого требуется вычислить (3–5 функций, в том числе и трансцендентные), из тех, которые предлагает программа.
- 3) Предусмотреть ввод исходных данных (границы интервала/начальное приближение к корню и погрешность вычисления) из файла или с клавиатуры по выбору конечного пользователя.
- 4) Выполнить верификацию исходных данных. Необходимо анализировать наличие корня на введенном интервале. Если на интервале несколько корней или они отсутствуют – выдавать соответствующее сообщение. Программа должна реагировать на некорректные введенные данные.
- 5) Для методов, требующих начальное приближение к корню (методы Ньютона, секущих, хорд с фиксированным концом, простой итерации), выбор начального приближения x_0 (а или b) вычислять в программе.
- 6) Для метода простой итерации проверять достаточное условие сходимости метода на введенном интервале.
- 7) Предусмотреть вывод результатов (найденный корень уравнения, значение функции в корне, число итераций) в файл или на экран по выбору конечного пользователя.
- 8) Организовать вывод графика функции, график должен полностью отображать весь исследуемый интервал.

Для систем нелинейных уравнений:

Метод простой итерации

- 1) Пользователь выбирает предлагаемые программой системы двух нелинейных уравнений (2-3 системы).
- 2) Организовать вывод графика функций.
- 3) Начальные приближения ввести с клавиатуры.
- 4) Для метода простой итерации проверить достаточное условие сходимости.
- 5) Организовать вывод вектора неизвестных: x_1, x_2 .
- 6) Организовать вывод количества итераций, за которое было найдено решение.
- 7) Организовать вывод вектора погрешностей: $|x_i^{(k)} - x_i^{(k-1)}|$
- 8) Проверить правильность решения системы нелинейных уравнений.

Основные этапы выполнения

Цель работы: Изучить численные методы решения нелинейных уравнений и систем нелинейных уравнений, а также выполнить программную реализацию методов.

Используемые формулы:

- Критерий окончания:

$$|x_{k+1} - x_k| < \varepsilon = 0.01$$

- Метод хорд:

- Рабочая формула:

$$x_{i+1} = x_i - \frac{(b - x_i) \cdot f(x_i)}{f(b) - f(x_i)}$$

- Условие выбора фиксированного конца:

$$f(b) \cdot f''(b) > 0 \rightarrow \text{зафиксировать конец } b$$

- Метод простых итераций:

- Преобразование уравнения:

$$x = \varphi(x) = x + \lambda \cdot f(x)$$

- Выбор параметра λ :

$$\lambda = \frac{1}{\max |f'(x)|}$$

- Условие сходимости:

$$|\varphi'(x)| \leq q < 1, \text{ где } q = \max |\varphi'(x)|$$

- Итерационная формула:

$$x_{k+1} = x_k + \lambda \cdot f(x_k)$$

- Метод секущих:

- Рабочая формула:

$$x_{i+1} = x_i - f(x_i) \cdot \frac{x_i - x_{i-1}}{f(x_i) - f(x_{i-1})}$$

- Метод Ньютона (для системы нелинейных уравнений):

- Матрица Якоби:

$$J(x, y) = \begin{pmatrix} \frac{df}{dx} & \frac{df}{dy} \\ \frac{dg}{dx} & \frac{dg}{dy} \end{pmatrix}$$

- Система приращений:

$$J(x_k, y_k) \cdot \begin{pmatrix} \Delta x \\ \Delta y \end{pmatrix} = - \begin{pmatrix} f(x_k, y_k) \\ g(x_k, y_k) \end{pmatrix}$$

- Обновление приближений:

$$x_{k+1} = x_k + \Delta x, \quad y_{k+1} = y_k + \Delta y$$

- Метод половинного деления:

- Рабочая формула:

$$x_i = \frac{a_i + b_i}{2}$$

- Приближенное значение корня:

$$x^* = \frac{a_n + b_n}{2} \text{ или } x^* = a_n \text{ или } x^* = b_n$$

- Метод Ньютона (для нелинейных уравнений):

- Рабочая формула:

$$x_i = x_{i-1} - \frac{f(x_{i-1})}{f'(x_{i-1})}$$

- Приближенное значение корня:

$$x^* = x_n$$

- Выбор начального приближения:

$$x_0 = \begin{cases} a_0, & f(a_0) \cdot f''(a_0) > 0 \\ b_0, & f(b_0) \cdot f''(b_0) > 0 \end{cases}$$

- Метод простых итераций (для системы нелинейных уравнений):

- Преобразование системы к эквивалентному виду:

$$\begin{cases} F_1(x_1, x_2, \dots, x_n) = 0 \\ F_2(x_1, x_2, \dots, x_n) = 0 \\ \dots \\ F_n(x_1, x_2, \dots, x_n) = 0 \end{cases} \rightarrow \begin{cases} x_1 = \varphi(x_1, x_2, \dots, x_n) \\ x_2 = \varphi(x_1, x_2, \dots, x_n) \\ \dots \\ x_n = \varphi(x_1, x_2, \dots, x_n) \end{cases}$$

$$\text{или } X = \varphi(X) \quad \varphi(X) = \begin{pmatrix} \varphi_1(X) \\ \varphi_2(X) \\ \dots \\ \varphi_n(X) \end{pmatrix}$$

- Итерационные формулы:

$$\begin{cases} x_1^{(k+1)} = \varphi_1(x_1^k, x_2^k, \dots, x_n^k) \\ x_2^{(k+1)} = \varphi_2(x_1^k, x_2^k, \dots, x_n^k) \\ \dots \\ x_n^{(k+1)} = \varphi_n(x_1^k, x_2^k, \dots, x_n^k) \end{cases}$$

- Достаточное условие сходимости:

$$\max_{x \in G} \|\varphi'(x)\| \leq q < 1$$

1. Вычислительная реализация задачи

Часть 1. Решение нелинейного уравнения

$$-2,7x^3 - 1,48x^2 + 19,23x + 6,35$$

1) Корни нелинейного уравнения графически

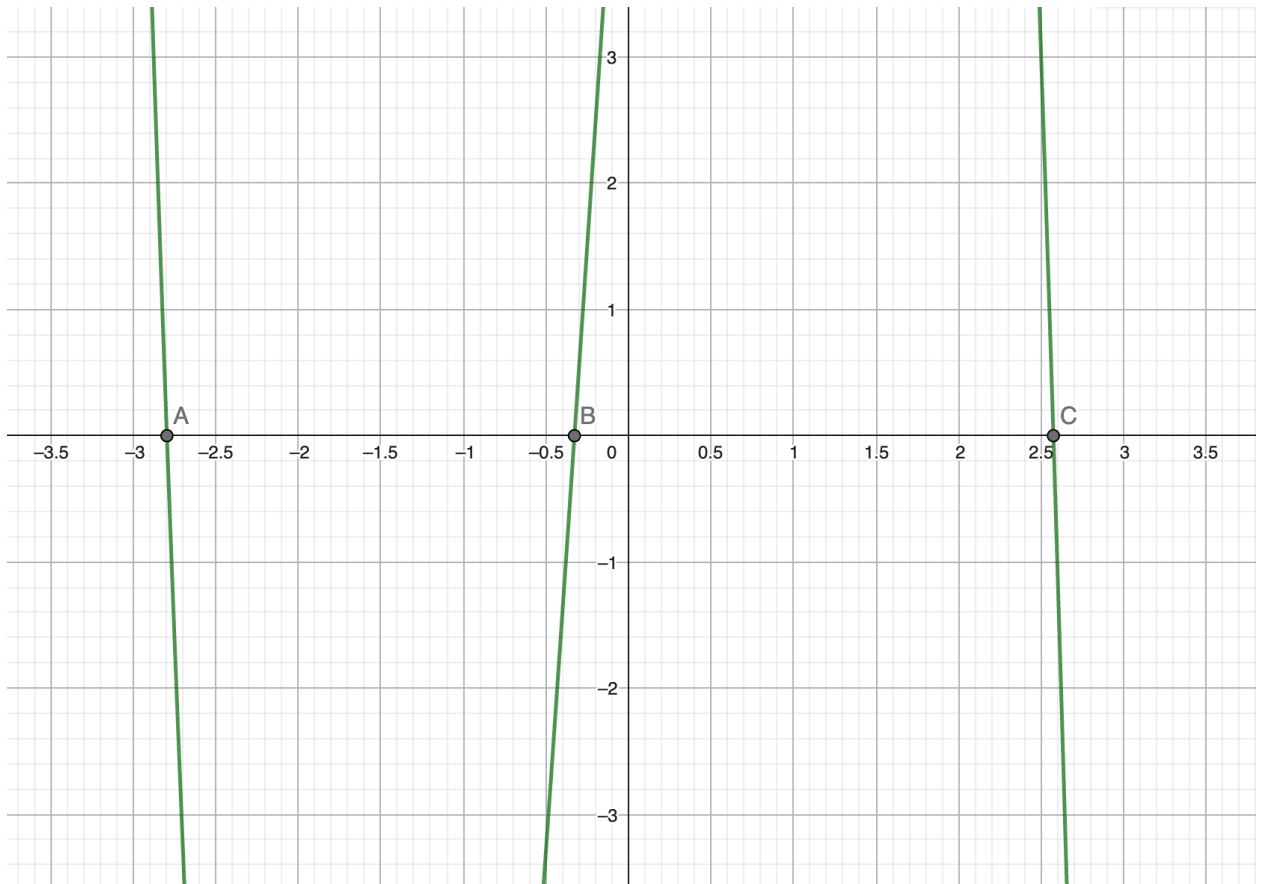


Рис. 1. График нелинейного уравнения $-2,7x^3 - 1,48x^2 + 19,23x + 6,35$

$$A(-2,79517; 0)$$

$$B(-0,32689; 0)$$

$$C(2,57392; 0)$$

2) Интервалы изоляции корней

Для определения интервалов изоляции корней данного уравнения можно воспользоваться методом интервалов знакопеременности. Для этого нужно найти значения функции на различных интервалах и определить знак функции на каждом из них.

- Приближенные значения корней:

$$x_1 \approx -2,8, x_1 \approx -0,3, x_1 \approx 2,6$$

- Интервалы:

$$(-\infty; -2,8), (-2,8; -0,3), (-0,3; 2,6), (2,6; +\infty)$$

- Определим знак функции. Для этого необходимо вычислить значения функции в произвольной точке каждого интервала.

1. Для $(-\infty; -2,8)$:

$$x = -3, \quad f(-3) = 8,24$$

2. Для $(-2,8; -0,3)$:

$$x = -1, \quad f(-1) = -11,66$$

3. Для $(-0,3; 2,6)$

$$x = 0, \quad f(0) = 6,35$$

4. Для $(2,6; +\infty)$

$$x = 3, \quad f(3) = -22,18$$

Таблица 1. Знаки $f(x)$ на каждом интервале.

$(-\infty; -2,8)$	$(-2,8; -0,3)$	$(-0,3; 2,6)$	$(2,6; +\infty)$
+	-	+	-

- Интервалы изоляции корней:

$$(-3; -2), (-1; 0), (2; 3)$$

- 3) Уточнение корней нелинейного уравнения с точностью $\varepsilon=10^{-2}$:

$$x_1 \approx -2,80, x_1 \approx -0,33, x_1 \approx 2,57$$

- 4) Методы уточнения корней:

- Крайний правый корень – Метод хорд

- Выбор фиксированного конца:

$$a = 2,000; f(a) = 17,29; f''(a) = -35,36 \rightarrow f(a) * f''(a) > 0 - \text{неверно}$$

$$b = 2,000; f(b) = -22,18; f''(a) = -51,56 \rightarrow f(b) * f''(b) > 0 - \text{верно}$$

Значит фиксируем конец b

Таблица 2. Уточнения корня методом хорд.

№ шага	a	b	x	$f(a)$	$f(b)$	$f(x)$	$ x_{k+1} - x_k $
1	2,000	3,000	2,438	17,290	-22,180	4,470	0,438
2	2,438	3,000	2,532	4,470	-22,180	0,916	0,094
3	2,532	3,000	2,551	0,916	-22,180	0,148	0,019

4	2,551	3,000	2,554	0,148	-22,180	0,024	0,003
---	-------	-------	--------------	-------	---------	-------	--------------

- *Крайний левый корень – Метод простых итераций*

- Проверка условия сходимости на интервале $(-3; -2)$

$$f(x) = -2,7x^3 - 1,48x^2 + 19,23x + 6,35 = 0$$

$$\varphi(x) = x + \lambda \cdot (-2,7x^3 - 1,48x^2 + 19,23x + 6,35)$$

$$f'(x) = -8,1x^2 - 2,96x + 19,23 = 0$$

$$f'(a) = -44,79 < 0; f'(b) = -7,25 < 0$$

$$\max(|f'(a)|, |f'(b)|) = 44,79 \rightarrow \lambda = \frac{1}{44,79}$$

$$\varphi(x) = x + \frac{1}{44,79} \cdot (-2,7x^3 - 1,48x^2 + 19,23x + 6,35)$$

$$\varphi'(x) = 1 + \frac{1}{44,79} (-8,1x^2 - 2,96x + 19,23)$$

$$\varphi'(-3) = -0,099, \varphi'(-2) = 0,772$$

$$q = \max |\varphi'(x)| \rightarrow q = 0,772 < 1$$

Значит условие сходимости $|\varphi'(x)| \leq q < 1$ выполняется.

Таблица 3. Уточнения корня методом простых итераций.

№ шага	x_k	x_{k+1}	$f(x_{k+1})$	$ x_{k+1} - x_k $
1	-3,000	-2,816	8,240	0,184
2	-2,816	-2,799	0,143	0,017
3	-2,799	-2,796	0,028	0,003

- *Центральный корень – Метод секущих*

Таблица 4. Уточнение корня методом секущих.

№ шага	x_{k-1}	x_k	x_{k+1}	$f(x_{k+1})$	$ x_{k+1} - x_k $
1	-1,000	0,000	-0,373	-0,889	0,373
2	0,000	-0,373	-0,327	0,936	0,046
3	-0,373	-0,327	-0,351	0,012	0,024

4	-0,327	-0,351	-0,352	-0,002	0,001
---	--------	--------	---------------	--------	--------------

Часть 2. Решение системы нелинейных уравнений

$$\begin{cases} \operatorname{tg}(xy + 0,3) = x^2 \\ 0,9x^2 + 2y^2 = 1 \end{cases}$$

Метод Ньютона

1) Корни системы нелинейных уравнений графически

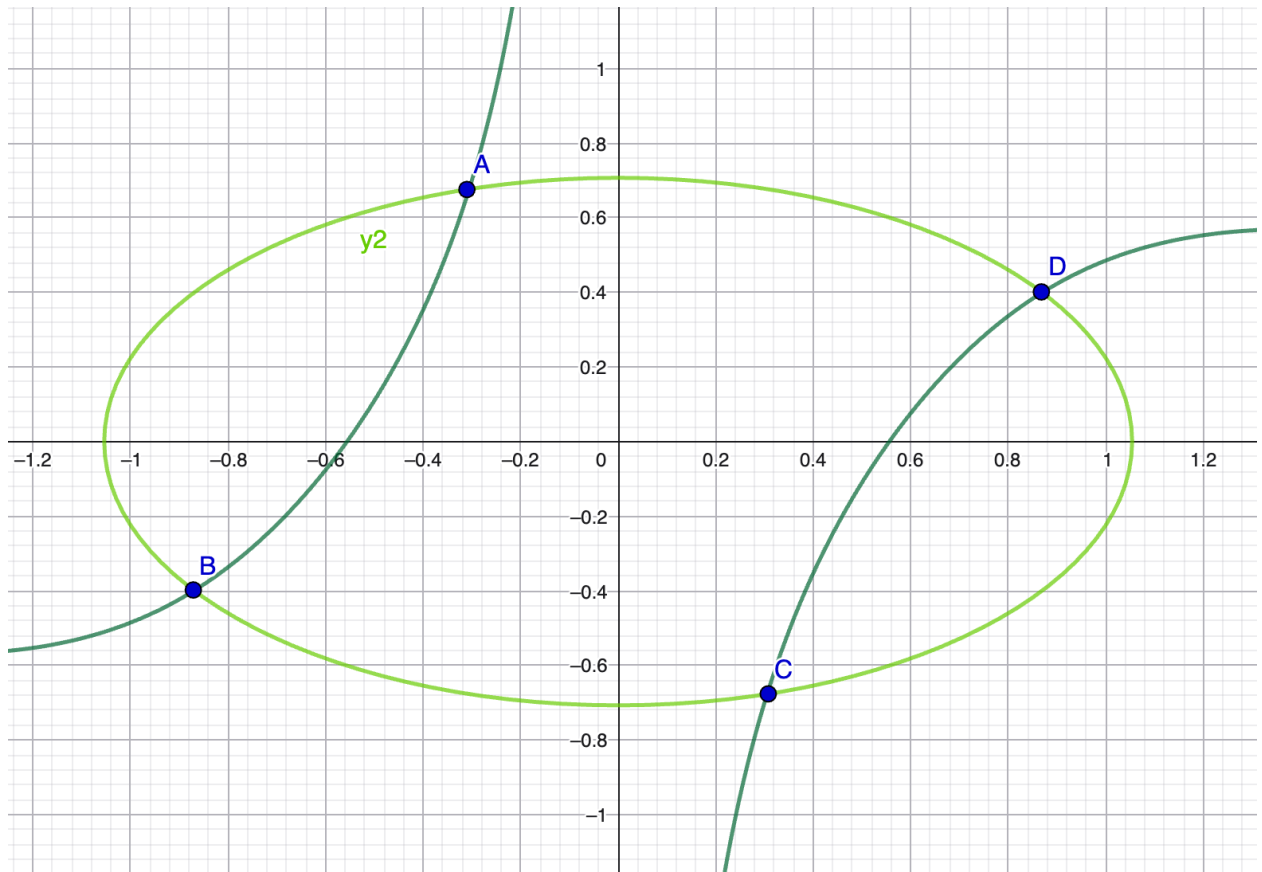


Рис. 1. График системы нелинейных уравнений $\begin{cases} \operatorname{tg}(xy + 0,3) = x^2 \\ 0,9x^2 + 2y^2 = 1 \end{cases}$

A (-0,305435; 0,676771)

B (-0,815678; -0,447885)

C (0,305435; -0,676771)

D (0,815678; 0,447885)

2) Решение системы нелинейных уравнений методом Ньютона с точность до 0,01

$$\begin{cases} f(x, y) = \operatorname{tg}(xy + 0,3) - x^2 = 0 \\ g(x, y) = 0,9x^2 + 2y^2 - 1 = 0 \end{cases}$$

▪ Матрица Якоби:

$$J(x, y) = \left(\frac{y}{\cos^2(xy + 0.3)} - 2x \quad \frac{x}{\cos^2(xy + 0.3)} \right)_{1,8x \quad 4y}$$

▪ Система:

$$\begin{cases} \left(\frac{y}{\cos^2(xy + 0.3)} - 2x \right) \Delta x + \left(\frac{x}{\cos^2(xy + 0.3)} \right) \Delta y = -f(x, y) \\ (1,8x)\Delta y + (4y)\Delta x = -g(x, y) \end{cases}$$

▪ Начальное приближение: $x_0 = -0,3$; $y_0 = 0,7$

- Итерация 1

$$\begin{cases} 1,306\Delta x - 0,302\Delta y = -0,0002 \\ -0,54\Delta x + 2,8\Delta y = 0,061 \end{cases} \rightarrow \Delta x = -0,0054, \Delta y = -0,0228$$

$$x_1 = -0,3 + (-0,0054) = -0,3054$$

$$y_1 = 0,7 + (-0,0228) = 0,6772$$

$$|\Delta x| = 0,0054 < 0,01 - \text{неверно}$$

$$|\Delta y| = 0,0228 < 0,01 - \text{неверно}$$

- Итерация 2

$$\begin{cases} 1,2939\Delta x - 0,3081\Delta y = -0,0002 \\ -0,5497\Delta x + 2,7088\Delta y = 0,0012 \end{cases} \rightarrow \Delta x = -0,0003, \Delta y = -0,0005$$

$$x_1 = -0,3054 + (-0,0003) = -0,3057$$

$$y_1 = 0,6772 + (-0,0005) = 0,6767$$

$$|\Delta x| = 0,0003 < 0,01 - \text{верно}$$

$$|\Delta y| = 0,0005 < 0,01 - \text{верно}$$

Таким образом, с помощью метода Ньютона нашлась точка $A (-0,3057; 0,6767)$. В силу симметрии системы уравнений относительно начала координат, точка C получается из точки A заменой знаков координат:

$$(x, y) \rightarrow (-x, -y) \rightarrow C(0,3057; -0,6767)$$

▪ Начальное приближение: $x_0 = -0,8$; $y_0 = 0,45$

▪ Итерация 1

$$\begin{cases} 0,8771\Delta x - 1,2851\Delta y = -0,1354 \\ -1,44\Delta x - 1,8\Delta y = 0,019 \end{cases}$$

$$\rightarrow \Delta x = -0,0157, \Delta y = -0,0023$$

$$x_1 = -0,8 + (-0,0157) = -0,8157$$

$$y_1 = -0,45 + (-0,0023) = -0,4477$$

$$|\Delta x| = 0,0157 < 0.01 \text{ — неверно}$$

$$|\Delta y| = 0,0023 < 0.01 \text{ — неверно}$$

▪ Итерация 2

$$\begin{cases} 0,9041\Delta x - 1,3251\Delta y = -0,1183 \\ -1,4683\Delta x + 1,7908\Delta y = 0,0003 \end{cases} \rightarrow \Delta x = -0,0000, \Delta y = -0,0002$$

$$x_1 = -0,8157 + (-0,0000) = -0,8157$$

$$y_1 = -0,4477 + (-0,0002) = -0,4479$$

$$|\Delta x| = 0,0000 < 0.01 \text{ — верно}$$

$$|\Delta y| = 0,0002 < 0.01 \text{ — верно}$$

Таким образом, с помощью метода Ньютона нашлась точка B (-0,8157; -0,4479). В силу симметрии системы уравнений относительно начала координат, точка D получается из точки B заменой знаков координат:

$$(x, y) \rightarrow (-x, -y) \rightarrow D(0,8157; 0,4479)$$

2. Программная реализация

Часть 1. Нелинейные уравнения

- Метод половинного деления, Метод Ньютона, Метод простых итераций

▪ Листинг кода (Основные методы)

```
def half_division_method(func, a, b, epsilon,
max_iterations=1000):
    fa = func(a)
    fb = func(b)

    if fa * fb > 0:
        raise ValueError(f"На интервале [{a}, {b}] нет корня
(знаки на концах одинаковы)")

    iteration = 0
    while (b - a) > epsilon and iteration < max_iterations:
        iteration += 1
        x = (a + b) / 2
        fx = func(x)

        if abs(fx) < epsilon:
            return {'root': x, 'f_root': fx, 'iterations':
iteration}

        if fa * fx < 0:
            b = x
            fb = fx
        else:
            a = x
            fa = fx

    x_final = (a + b) / 2
    return {'root': x_final, 'f_root': func(x_final),
```

```

'iterations': iteration}

def newton_method(func, deriv_func, x0, epsilon,
max_iterations=1000):
    x = x0
    iteration = 0

    while iteration < max_iterations:
        iteration += 1
        fx = func(x)
        dfx = deriv_func(x)

        if abs(dfx) < 1e-12:
            raise ValueError(f"Производная близка к нулю
(f'({x:.4f})={dfx:.4e}). Метод не сходится")

        x_new = x - fx / dfx

        if abs(x_new - x) < epsilon:
            return {'root': x_new, 'f_root': func(x_new),
'iterations': iteration}

        x = x_new

    raise ValueError(f"Не достигнута точность за
{max_iterations} итераций. Последнее x={x}")

def simple_iteration_method(func, deriv_func, a, b, epsilon,
x0, max_iterations=1000):
    n_samples = 1000
    x_samples = np.linspace(a, b, n_samples)

    deriv_values = [deriv_func(x) for x in x_samples]
    abs_deriv_values = [abs(d) for d in deriv_values]

    M = max(abs_deriv_values)

    if M == 0:
        raise ValueError("Производная равна нулю на
интервале. Метод неприменим")

    mid = (a + b) / 2
    sign_df = 1 if deriv_func(mid) >= 0 else -1
    lambda_val = -sign_df / M

    phi_deriv_values = [abs(1 + lambda_val * d) for d in
deriv_values]
    q = max(phi_deriv_values)

    if q >= 1:
        raise ValueError(f"\nДостаточное условие сходимости
не выполняется (q = {q:.4f} >= 1)")

    x = x0
    iteration = 0

    while iteration < max_iterations:
        iteration += 1
        x_new = x + lambda_val * func(x)

        if abs(x_new - x) < epsilon:
            return {'root': x_new, 'f_root': func(x_new),

```

```

'iterations': iteration}

    x = x_new

(f"Не достигнута точность за {max_iterations} итераций.
Последнее x={x}")
■

def choose_newton_start(func, second_deriv_func, a, b):

    fa = func(a)
    fb = func(b)
    d2fa = second_deriv_func(a)
    d2fb = second_deriv_func(b)

    if fa * d2fa > 0:
        return a
    elif fb * d2fb > 0:
        return b
    else:
        return (a + b) / 2

def choose_iteration_start(func, a, b):
    if abs(func(a)) < abs(func(b)):
        return a
    else:
        return b

def check_root(func, a, b, num_points=100):
    x_values = np.linspace(a, b, num_points)
    y_values = [func(x) for x in x_values]

    sign_changes = 0
    for i in range(len(y_values) - 1):
        if y_values[i] * y_values[i + 1] < 0:
            sign_changes += 1
        elif abs(y_values[i]) < 1e-10:
            sign_changes += 1

    if sign_changes == 0:
        return False, 0, "Корней на интервале не
обнаружено"
    elif sign_changes > 1:
        return False, sign_changes, f"На интервале
обнаружено {sign_changes} корней (рекомендуется сузить
интервал)"
    else:
        return True, sign_changes, "Обнаружен 1 корень"

```

- Результат работы программы

=====

ДОСТУПНЫЕ УРАВНЕНИЯ:

=====

1. $f(x) = -1.5x^3 + 2.3x^2 + 4.1x - 3.7$

2. $f(x) = x^3 - 2.15x^2 - 1.8x + 2.4$

3. $f(x) = e^x - 2.5x - 1.2$

4. $f(x) = \sin(x) + 0.5x - 1.3$

=====

Выберите номер уравнения (1-4): 1

Выберите метод решения:

1. Метод половинного деления

2. Метод Ньютона

3. Метод простых итераций

Ваш выбор (1-3): 2

Выберите источник ввода данных:

1. С клавиатуры

2. Из файла

Ваш выбор (1-2): 2

Введите имя файла: 1.txt

=====

ИСХОДНЫЕ ДАННЫЕ

=====

Выбрано: $f(x) = -1.5x^3 + 2.3x^2 + 4.1x - 3.7$

Интервал изоляции: [2.0, 3.0]

Точность: $\varepsilon = 0.001$

=====

Проверка наличия корня на интервале

Обнаружен 1 корень

Автоматически выбрано начальное приближение (Ньютон): $x_0 = 3.0000$

Куда вывести результаты?

1. На экран

2. В файл

Ваш выбор (1-2): 1

РЕЗУЛЬТАТЫ РЕШЕНИЯ

Уравнение: $-1.5x^3 + 2.3x^2 + 4.1x - 3.7$

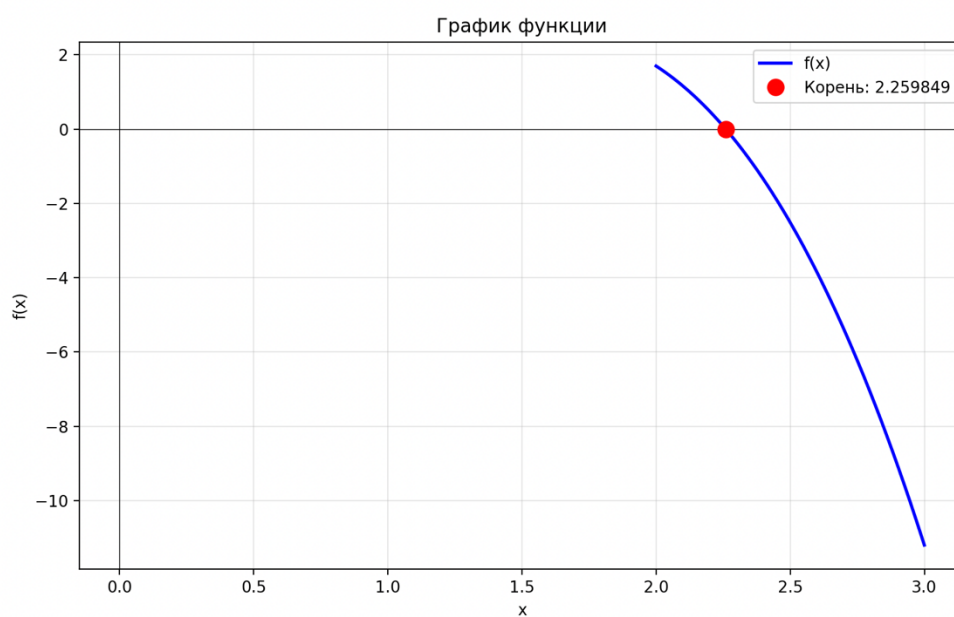
Метод: Метод Ньютона

Найденный корень: $x = 2.2598487100$

Значение функции: $f(x) = -0.0000000000$

Число итераций: 5

Показать график функции? (y/n): y



Часть 2. Системы нелинейных уравнений

- Метод простой итерации

■ Листинг кода (Основные методы)

```
def simple_iteration(g1, g2, x1, x2, eps, max_iter=1000):
    iteration = 0
    while iteration < max_iter:
        iteration += 1
        x1_new = g1(x1, x2)
        x2_new = g2(x1, x2)
        if max(abs(x1_new - x1), abs(x2_new - x2)) < eps:
            return {'root': (x1_new, x2_new),
                    'iterations': iteration}
        x1, x2 = x1_new, x2_new
    raise ValueError("Точность не достигнута за максимальное число итераций")

def check(system, x1, x2):
    row1 = abs(system['dg1dx1'](x1, x2)) +
    abs(system['dg1dx2'](x1, x2))
    row2 = abs(system['dg2dx1'](x1, x2)) +
    abs(system['dg2dx2'](x1, x2))
    q = max(row1, row2)
    return q < 1, q
```

■ Результат работы программы

ДОСТУПНЫЕ СИСТЕМЫ:

1.

$$x1 - 0.5\cos(x2) - 0.5 = 0$$

$$x2 - 0.5\sin(x1) - 0.5 = 0$$

2.

$$x1 - \sin(x2)/2 - 1 = 0$$

$$x2 - \cos(x1)/2 - 1 = 0$$

Выберите систему (1-2): 1

Выберите источник ввода данных:

1. С клавиатуры

2. Из файла

Ваш выбор(1-2): 2

Введите имя файла: 2.txt

Проверка условия сходимости

$$q = 0.4388$$

Проверка правильности решения

Решение верное

Куда вывести результаты?

1. На экран

2. В файл

Ваш выбор(1-2): 1

РЕЗУЛЬТАТЫ РЕШЕНИЯ

Система:

$$x_1 - 0.5\cos(x_2) - 0.5 = 0$$

$$x_2 - 0.5\sin(x_1) - 0.5 = 0$$

Коэффициент сходимости: $q = 0.438791$

Найденный корень:

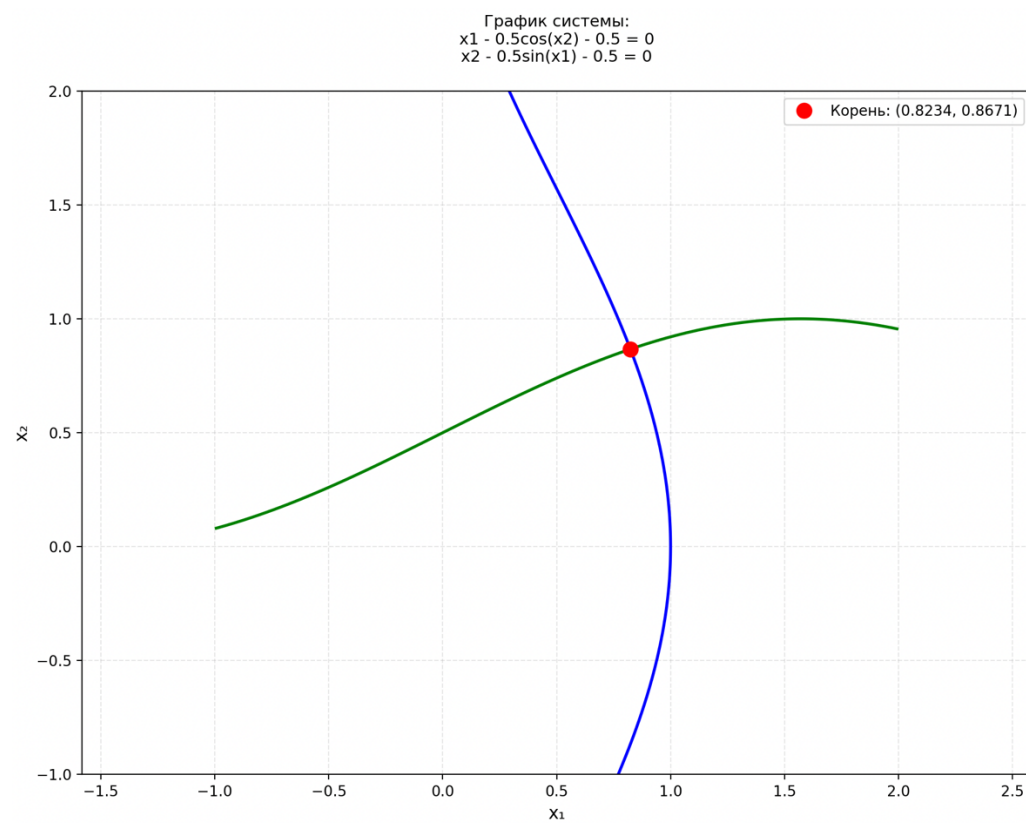
$$x_1 = 0.823392324379$$

$$x_2 = 0.867065106412$$

Число итераций: 7

Показать график системы?

Ваш выбор (y/n): y



Вывод

Выполнение лабораторной работы по вычислительной математике позволило мне познакомиться с методами для уточнения корня НУ и СНУ. Данная работа дала мне возможность применить полученные теоретические знания на практике, развить навыки работы с информацией. В процессе выполнения я реализовала алгоритм 3х методов уточнения корня (Метод половинного деления, метод Ньютона, метод простых итераций) на ЯП Python. Приобретённый опыт поможет мне при дальнейшем изучении предмета вычислительная математика.